

Практическая работа 10

АННОТАЦИИ, РЕФЛЕКСИЯ И DYNAMIC PROXY

Цель работы

Освоить runtime-аннотации, рефлексия и динамический проху для наблюдения за прикладными операциями без переноса бизнес-логики.

Описание программы

Публичные операции сервиса помечаются runtime-аннотацией. Рефлексия находит метаданные, а Проху перехватывает вызовы интерфейса, считает отмеченные операции и передаёт их делегату.

Общие сведения

Аннотация описывает назначение операции. InvocationHandler различает отмеченные и обычные методы, обновляет метрики и разворачивает InvocationTargetException, сохраняя исходную ошибку.

Функциональное назначение и ограничения

Наблюдение даёт количество вызовов и имена операций, не меняя результата сервиса. Неаннотированный all пропускается.

Проху работает через интерфейс сервиса. Метрики не заменяют доменные проверки, а исходное исключение не должно уходить наружу в оболочке рефлексии.

Логическая структура

При вызове handler ищет соответствующий Method, читает аннотацию, фиксирует метрику и выполняет invoke. Cause отражённой ошибки выбрасывается без изменения типа.

Используемые технические средства

Java Reflection API, java.lang.reflect.Proxy, runtime-аннотации, JUnit и Gradle.

Входные и выходные данные

Входные данные. Вызовы addPlan, changeStatus и all через прокси-ссылку.

Выходные данные. Совпадающие результаты прямого и proxied-вызова, счётчики операций и исходные исключения.

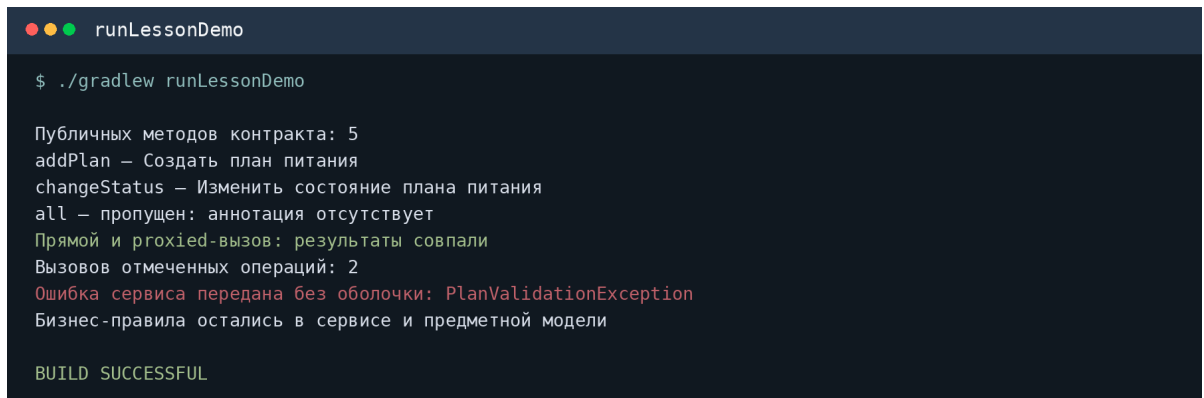
Контрольные сценарии

Компонент	Ответственность	Проверка
Пользовательский интерфейс	Ввод и обратная связь	UI-снимки
Прикладная логика	Сценарии и правила	Автотесты
Данные/инфраструктура	Состояние и хранение	Протокол запуска

Ход работы

Проект собран и проверен по командам из README и протокола проверки. Ниже приведены фактические фрагменты реализации и результаты запусков; учебные изображения преподавателя не использовались.

1. Демонстрация механизма. Фактическое состояние получено при запуске проекта и сохранено в каталоге docs/evidence. Оно подтверждает соответствующий пользовательский сценарий и отсутствие аварийного завершения.



```
runLessonDemo

$ ./gradlew runLessonDemo

Публичных методов контракта: 5
addPlan – Создать план питания
changeStatus – Изменить состояние плана питания
all – пропущен: аннотация отсутствует
Прямой и proxied-вызов: результаты совпали
Вызовов отмеченных операций: 2
Ошибка сервиса передана без оболочки: PlanValidationException
Бизнес-правила остались в сервисе и предметной модели

BUILD SUCCESSFUL
```

Рисунок 1 – Рефлексия, метрики и передача ошибки

2. Интерфейс приложения. Фактическое состояние получено при запуске проекта и сохранено в каталоге docs/evidence. Оно подтверждает соответствующий пользовательский сценарий и отсутствие аварийного завершения.

Аннотации, reflection и наблюдение за вызовами
Планов: 1

[illegible]

Поля сохраняются после ошибки; пример
даты: 2026-11-01

Название плана

Дата начала (ГГГГ-ММ-ДД)

2026-10-01

Состояние

Черновик

Сохранить план

Действия с выбранной строкой

Сделать активным

Удалить выбранный

Заполните поля. Ошибка не удалит
введённые значения

3. Стабильный результат. Фактическое состояние получено при запуске проекта и сохранено в каталоге docs/evidence. Оно подтверждает соответствующий пользовательский сценарий и отсутствие аварийного завершения.

Аннотации, reflection и наблюдение за вызовами
Планов: 3

[illegible]

Поля сохраняются после ошибки; пример
даты: 2026-11-01

2026-12-01

Черновик ▼

Сохранить план

Сделать активным

Сделать активным

Удалить выбранный

План «Снижение массы» теперь действует

Вывод

Runtime-аннотации и dynamic proxy добавили наблюдаемость без дублирования бизнес-правил. Отмеченные операции подсчитываются, обычные методы не искажаются, а исходные исключения передаются прозрачно.

Студент группы 090302-ИСТа-023

Карионов В.В.

Преподаватель

Елкин Н.С.